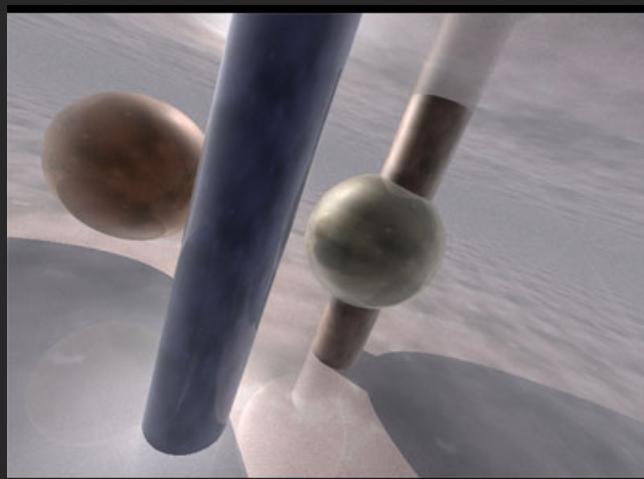


Inigo Quilez :: articles :: simple gpu raytracing - 2005



Prefade

When you first try to code for a new platform that provides a pixel buffer that you can write to, there are only two types of "Hello World" applications you can write - either a Mandelbrot set or a small raytracer/raymarcher. Pixel shaders 3.0 was such a platform for me in 2005, cause it was the first shader model that allows for conditional branching. It was still not as flexible as regular CPU programing or current GPU shaders, but it was good enough to try raytracing a few sphere in it. 15 years later, in a workd of abudant shader based raytracers and raymarchers (hello Shadertoy!) this feels obvious, but in 2005 it was all new territory. So, this article might seem terrible simply by today standards, but I leave it here for historical context. The rest of the article will be written in present tense.



The idea

So, it seems GPUs are so fast these days that even a brute force implementations of raytracing runs fast at high screen resolutions. This is perfect for tiny demos (like the 4 kilobyte demos of the Demoscene) where you don't really have room for acceleration structures or anything. After a few months playing with this tech I've finally made a little demo called *Kinderpainter* as an experiment on GLSL coding. I decided to implement a simple Whitted raytracer: only local lighting plus one shadow and one perfectly specular reflection. I used two spheres, two cylinders and two planes as scene, and I allowed them to move so I could build two or three virtual different scenes and so I could synchronize the movements to the music too. All the image was synthesized in a quad covering the complete screen to which a pixel shader was attached. The shader was responsible for creating the image, and the CPU was just left with the code to create a desktop window, initialize OpenGL and the shader, move the objects with some trigonometric functions, and render the quad.

As a simple raytracer, what the code does is, for each pixel, cast a ray on the scene to find the closest intersected object. That's done in the *calcinter()* function on the pixel shader below. The implementation just calls the intersection functions for the six objects in the scene (two spheres, two cylinders and two planes), while it keeps track of the closest intersection at all time.

Then the code calls the shading function *calcshade()*. The first thing this one does is to compute the normal of the object at the intersection point, by calling *calcnor()*. Depending on the primitive type, that function executes the necessary computations. Then *calcshade()* does some basic diffuse and specular lighting calculations. It also calls *calcshadow* to decide if the point being shaded is in shadow or not. This function is a simplified version of the regular intersection function *calcinter()* (it's simple cause we don't really need to know the closest intersected object, we just need to know if *any* object was intersected at all).

Therefore *main()* function, the one executed for each pixel, just calls *calcinter()* and *calcshade()* and returns the result to the hardware so the pixel of the screen is colored. Normally a Whitted raytracer would recursively call *calcinter()* and *calcshade()* from within the *calcshade()* function, up to a number of levels, say 4. However, since current GPU shader models don't support recursion yet, I made the trick of doing a nonrecursive version of *calcshade()* and calling *calcinter()* and *calcshade()* for a second time from the *main()* function, with the right reflection ray as argument.

The complete pixel shader is below, and you have a live version online in [Shadertoy](#). Of course, *fpar00[]* contains all the input to the shader. For example, *fpar00[0]* contains the information of the first sphere (a position and a radius), *fpar00[1]* for the second sphere, *fpar00[2]* and *fpar00[3]* are the two cylinders, and *fpar00[4]* and *fpar00[5]* are the two planes. The colors of the objects are stored from *fpar00[6]* to *fpar00[11]*. Also, *fpar00[12]* contains the camera position, and *fpar00[13]*, *fpar00[14]* and *fpar00[15]* contain the camera matrix. The raydirection is partially computed in the vertex shader for the corners of the full screen quad, and interpolated down to the pixel shader by the rasterization hardware, and it arrives to the pixel shader thru the

varying called *raydir*.

The Code

Pixel shader:

```
uniform vec4 fpar00[16];
uniform sampler2D tex00;
varying vec3 raydir;

bool intSphere( in vec4 sp, in vec3 ro, in vec3 rd, in float tm, out float t )
{
    bool r = false;
    vec3 d = ro - sp.xyz;
    float b = dot(rd,d);
    float c = dot(d,d) - sp.w*sp.w;
    t = b*b-c;
    if( t > 0.0 )
    {
        t = -b-sqrt(t);
        r = (t > 0.0) && (t < tm);
    }

    return r;
}

bool intCylinder( in vec4 sp, in vec3 ro, in vec3 rd, in float tm, out float t )
{
    bool r = false;
    vec3 d = ro - sp.xyz;
    float a = dot(rd.xz,rd.xz);
    float b = dot(rd.xz,d.xz);
    float c = dot(d.xz,d.xz) - sp.w*sp.w;
    t = b*b - a*c;
    if( t > 0.0 )
    {
        t = (-b-sqrt(t)*sign(sp.w))/a;
        r = (t > 0.0) && (t < tm);
    }
    return r;
}

bool intPlane( in vec4 pl, in vec3 ro, in vec3 rd, in float tm, out float t )
{
    t = -(dot(pl.xyz,ro)+pl.w)/dot(pl.xyz,rd);
    return (t > 0.0) && (t < tm);
}
```

```
}

vec3 calcnor(in vec4 ob,in vec4 ot,in vec3 po,out vec2 uv )
{
    vec3 no;

    if(ot.w>2.5)
    {
        no.xz = po.xz-ob.xz;
        no.y = 0.0;
        no = no/ob.w;
        uv = vec2(no.x,po.y+fpar00[12].w);
    }
    else if(ot.w>1.5)
    {
        no = ob.xyz;
        uv = po.xz*.2;
    }
    else
    {
        no = po-ob.xyz;
        no = no/ob.w;
        uv = no.xy;
    }

    return no;
}

float calcinter(in vec3 ro,in vec3 rd,out vec4 ob,out vec4 co)
{
    float tm=10000.0;
    float t;

    if( intSphere( fpar00[0],ro,rd,tm,t) ) { ob = fpar00[0]; co = fpar00[ 6]; tm
    if( intSphere( fpar00[1],ro,rd,tm,t) ) { ob = fpar00[1]; co = fpar00[ 7]; tm
    if( intCylinder(fpar00[2],ro,rd,tm,t) ) { ob = fpar00[2]; co = fpar00[ 8]; tm
    if( intCylinder(fpar00[3],ro,rd,tm,t) ) { ob = fpar00[3]; co = fpar00[ 9]; tm
    if( intPlane( fpar00[4],ro,rd,tm,t) ) { ob = fpar00[4]; co = fpar00[10]; tm
    if( intPlane( fpar00[5],ro,rd,tm,t) ) { ob = fpar00[5]; co = fpar00[11]; tm

    return tm;
}

bool calcshadow(in vec3 ro,in vec3 rd,in float l)
{
    float t;

    bvec4 ss = bvec4(intSphere( fpar00[0],ro,rd,l,t),
```

```

        intSphere( fpar00[1],ro,rd,l,t),
        intCylinder(fpar00[2],ro,rd,l,t),
        intCylinder(fpar00[3],ro,rd,l,t));

    return any(ss);
}

vec4 calcshade(in vec3 po,in vec4 ob,in vec4 co,in vec3 rd,out vec4 re)
{
    float di,sp;
    vec2 uv;
    vec3 no;
    vec4 lz;

    lz.xyz = vec3(0.0,1.5,-3.0) - po;
    lz.w = length(lz.xyz);
    lz.xyz = lz.xyz/lz.w;

    no = calcnor(ob,co,po,uv);

    di = dot(no,lz.xyz);
    re.xyz = reflect(rd,no);
    sp = dot(re.xyz,lz.xyz);
    sp = max(sp,0.0);
    sp = sp*sp;
    sp = sp*sp;

    if( calcshadow(po,lz.xyz,lz.w) )
        di = 0.0;

    di = max(di,0.0);
    co = co*texture2D(tex00,uv)*(vec4(.21,.28,.3,1) + .5*vec4(1.0,.9,.65,1.0)*di)

    di = dot(no,-rd);
    re.w = di;
    di = 1.0-di*di;
    di = di*di;

    return(co+0.6*vec4(di));
}

void main( void )
{
    float tm;
    vec4 ob,co,co2,re,re2;
    vec3 no,po;
    vec3 ro = fpar00[12].xyz;
    vec3 rd = normalize(raydir);

    tm = calcinter(ro,rd,ob,co);

```

```
po = ro + rd*tm;
co = calcshade(po,ob,co,rd,re);

tm = calcinter(po,re.xyz,ob,co2);
po += re.xyz*tm;
co2 = calcshade(po,ob,co2,re.xyz,re2);
gl_FragColor=mix(co,co2,.5-.5*re.w) + vec4(fpar00[13].w);
};
```

Vertex shader:

```
varying vec3 raydir;
uniform vec4 fpar00[16];

void main( void )
{
    vec3 r;

    gl_Position=gl_Vertex;

    r = gl_Vertex.xyz*vec3(1.3333,1.0,0.0)+vec3(0.0,0.0,-1.0);

    raydir.x=dot(r,fpar00[13].xyz);
    raydir.y=dot(r,fpar00[14].xyz);
    raydir.z=dot(r,fpar00[15].xyz);
};
```

inigo quilez - learning computer graphics since 1994